Yes — with `filepath.py`, the full picture is clear. These classes are specifically designed to **watch the filesystem locations of configured certificate/key files and react when one of those watched files is created**.

`FilepathWatcher` provides the missing link: it takes each configured certificate/key filepath, extracts its parent directory, and tells the `watchdog.Observer` to watch that directory:

```
parents = [Path(p).resolve().parent for p in self.filepaths]

for directory_path in parents:
    self.observer.schedule(
        self.handler,
        path=directory_path,
```

```
        recursive=recursive
    )
```

So if your configured identity points to:

```
/certs/my-cert.pem
/certs/my-key.pem
```

it watches:

```
/certs/
```

while `CertificateHandler` filters the events so it only cares about those specific configured files.

The complete flow is essentially:

```
startup()
   │
   ├─ CertificateWatcher()
   │       │
   │       ├─ reads configuration.IDENTITIES
   │       │
   │       ├─ gets cert paths + key paths
   │       │
   │       └─ creates CertificateHandler
   │
   ├─ cw.start(recursive=False)
   │       │
   │       ├─ gets parent directories
   │       │
   │       ├─ Observer.schedule(...)
   │       │
   │       └─ Observer.start()
   │
   │           filesystem event
   │                 │
   │                 ▼
   │       CertificateHandler.on_created()
   │                 │
   │                 ├─ identifies certificate
   │                 ├─ validate_certificate()
   │                 └─ adds Event to queue
   │
   └─ cw.watch(send_to_event_service)
                     │
                     ▼
```

```
        send_to_event_service()
                 │
                 ├─ sends event to system queue
                 ├─ update_certificate_services()
                 └─ broadcasts /events
```

But there is an important limitation

Now that you've shown all four pieces, I can confirm that **this code does not generically detect every certificate change**.

It specifically reacts to:

Python Run

There is no:

```python
def on_modified(...)
def on_moved(...)
def on_deleted(...)
```

So the watcher is monitoring the directory continuously, but the certificate-specific handler only does something when `watchdog` classifies the event as a **creation**.

For example, suppose this already exists:

```
/certs/client.crt
```

If an application opens it and rewrites its contents:

```
client.crt
    ↓
contents modified
```

`watchdog` will normally produce a modification event, which eventually corresponds to:

Python Run

Your `CertificateHandler` hasn't implemented that method, so **nothing certificate-specific happens**.

Conversely, if:

```
/certs/client.crt
```

doesn't exist and you create it, you should get:

```
Python
```

Run

and your code performs the validation and refresh.

Atomic certificate replacement is another concern

A lot of software updates files atomically using something conceptually like:

```
client.crt.tmp
      |
      └── rename/replace ──> client.crt
```

That can generate a `moved` event rather than a `created` event.

Your code wouldn't process that either because you don't have:

```
Python
```

Run

So I'd describe the current implementation as:

> **A watcher for the configured certificate/key paths that reacts when one of those paths receives a filesystem "created" event.**

I would *not* describe it as:

> **A watcher that reliably detects whenever a certificate changes.**

Those are slightly different things.

There's also a small issue in `FilepathWatcher`

This:

```
parents = [Path(p).resolve().parent for p in self.filepaths]

for directory_path in parents:
    self.observer.schedule(...)
```

doesn't deduplicate directories.

Suppose your cert and key are both here:

```
/certs/client.crt
/certs/client.key
```

You get:

```
parents = [
    Path("/certs"),
    Path("/certs"),
]
```

and potentially schedule the same handler against the same directory twice.

If there are several identities:

```
/certs/client1.crt
/certs/client1.key
/certs/client2.crt
/certs/client2.key
```

you could schedule `/certs` four times.

That could potentially result in duplicate event handling depending on how the observer handles those watches. It would be cleaner to deduplicate:

Python Run

One other Python issue I noticed

This is also worth changing:

```
class CertificateWatcher(FilepathWatcher):

    def __init__(self, queue: asyncio.Queue = asyncio.Queue()):
```

Using a mutable/runtime object as a default argument means the same `Queue` instance is created once and reused by every `CertificateWatcher()` created without an explicit queue.

A safer pattern is:

```
def __init__(self, queue: asyncio.Queue | None = None):
    if queue is None:
        queue = asyncio.Queue()
```

It's especially relevant with `asyncio` applications because queues and event loops can interact in unexpected ways if you create watchers in different contexts.

So, overall

Your architecture itself makes sense:

```
DirectoryWatcher
      ↑
FilepathWatcher
      ↑
CertificateWatcher
      |
CertificateHandler
```

Each layer gets more specialized:

```
DirectoryWatcher
    "Watch a directory"
```

```
FilepathWatcher
    "Watch directories containing these particular files"

CertificateWatcher
    "Those particular files are certificates/keys"

CertificateHandler
    "When one is created, validate it and generate a system event"
```

And then your lifespan code connects that event to:

Python Run

which appears to be the part responsible for **reacting at the application/service level to the changed certificate**.

The biggest thing I'd change if the intent really is **"detect whenever a certificate gets updated"** is the event handling, because right now `on_created()` alone is too narrow.

okay great. So what I want to do now is have a watcher that will register a new skill in a user directory. It should also re-register any skills that were updated or deleted. Below is the skill.py I already have. Can I reuse any of the previous code in order to do this? If not, what updates need to happen?:

```
import os

from dotenv import load_dotenv

from aidl import configuration
from aidl.connectors.gitlab.graphql import GitlabGraphQL
from aidl.models.api.agent import Skill, SkillAuthor
from aidl.models.api.registry import Registrant
from aidl.models.enum import RegistryType, SkillStatus
from aidl.registry.base import send_registration
```

```
def SKILL_CONTENT(path: str, group: str, project: str) -> str:
    return f"""
    query {{
    project(fullPath: "{group}/{project}) {{
        repository {{
        blobs(ref: "main", paths: [{path}/SKILL.md]) {{
            nodes {{
            name
            rawBlob
            }}
        }}
        }}
    }}
    }}
    """

def determine_authorship(project: dict) -> SkillAuthor:
    """
    Extract last editor metadata.

    Args:
        project: key/value pairs representing a SKILL
container.

    Returns:
        A dictionary of key/value pairs representing the last
editor of the SKILL.
    """
    return SkillAuthor(email="blah@bleh", name="Some
Person")

def extract_content(connection: GitlabGraphQL) -> str:
    """
    Extract SKILL md content.

    Args:
        connection: repository connector

    Returns:
        A string representing the content of the SKILL.
```

```
    """
    return """
    ---
    name: file_data_creator
    description: Create and save file data to virtual or local
filesystems.
    ---

    ### Instructions:
    1. When asked to create file data, use the specific
backend tool provided.
    2. Ensure all data is saved with correct encoding (default
to utf-8).
    3. Confirm the file path before saving.
    """

def extract_skills(query: str, token: str) -> list[tuple]:
    """
    Extract skills from storage.

    Args:
        query: search value to retreive from storage
        token: source storage auth token

    Returns:
        An array of tuples of (content, author, metadata,
project) representation of a SKILL.
    """

    # initialize
    gg = GitlabGraphQL(token=token)

    return [
        (
            extract_content(connection=gg),
            determine_authorship(p),
            parse_content(extract_content,p),
            p
        ) for p in gg.paginate(query,["projects"])
    ]
```

```
def parse_content(content: str, project) -> dict:
    """
    Extract metadata from SKILL content file.

    Args:
        content: SKILL content

    Returns:
        A dictionary of key/value pairs representing SKILL
metadata.
    """
    return {
        "description": project["description"],
        "name": project["name"]
    }

async def register_skills(registry_url: str, query: str,
user_env: str, token_key: str = None) -> list[Registrant]:
    """
    Register provided skills to gateway.

    Args:
        query: search value to retreive from storage
        registry_url: registry service location
        token_key: source storage auth token env key
        user_env: file path of environmental variable storage

    Returns:
        A list of dictionaries where each represents a SKILL
modeled as a registrant.
    """

    # reload .env in case edited after app init
    load_dotenv(user_env)

    # attempt to load token value
    token = os.getenv(token_key)

    # silently end
```

```
    if token is None: return list()

    # get from storage and structure
    # with validation and user preference
    skills = [
        (
            validate_skill(metadata, content),
            user_enabled(),
            Skill(
                content="blah",

description=metadata.get("description","Nothing
specified."),
                group=project["fullPath"],
                health_url="fill in from project",
                name=metadata.get("name","Unknown"),
                namespace=project["fullPath"],
                owners=[f"{configuration.BRAND_NAME} Team"
if project["fullPath"] in
configuration.SKILL_REPOSITORY_CORE_PATH else
(author.name if author else "Unknown")]
            )
        ) for content, author, metadata, project in
extract_skills(query, token)
    ]

    # register
    registrants = await send_registration(
        registry_url=registry_url,
        registrants=[
            Registrant(
                enables=skill.description,
                error=error,
                is_active=is_valid,
                is_core=skill.group in
configuration.SKILL_REPOSITORY_CORE_PATH,
                is_enabled=is_valid and is_enabled,
                namespace=skill.group,
                registration_key=skill.namespace,
                registry_location=f"
```

```
{skill.group}/{skill.name}/SKILL.md",
                registry_type=RegistryType.SKILL,
                source=skill,
                status=SkillStatus.AVAILABLE if is_valid else
SkillStatus.UNAVAILABLE
            ) for (is_valid,error), is_enabled, skill in skills
        ])

    return [Registrant(**r) for r in registrants]

def validate_skill(metadata: dict, content: str) -> tuple:
    return (True, None)

def user_enabled() -> bool:
    return True
```

Worked for 22s >

Yes. I would **reuse the generic** `DirectoryWatcher` / `DirectoryHandler` **architecture**, but I would **not reuse** `FilepathWatcher` for skills.

The reason is the difference in what you're watching:

```
Certificates:
Known files ahead of time
    /certs/user.crt
    /certs/user.key
        ↓
FilepathWatcher makes sense


Skills:
Known root directory
    /users/selvyn/skills/
        ├── calculator/
        │   └── SKILL.md
        ├── weather/
        │   └── SKILL.md
        └── brand-new-skill/   <-- can appear later
            └── SKILL.md
        ↓
DirectoryWatcher makes more sense
```

That also gives you exactly what you want: because `watchdog` watches the root recursively, **you do not have to restart the watcher when a new skill directory appears**.

What I would reuse

Your hierarchy could become:

```
DirectoryWatcher
    │
    ├── FilepathWatcher
    │       └── CertificateWatcher
    │
    └── SkillWatcher
```

So certificates continue working the way they do today, while skills use the same base infrastructure.

I would create something along these lines:

```
aidl/filesystem/
    directory.py       <-- existing generic watcher
    filepath.py        <-- existing certificate support

aiidentity/
    certificate/
        store.py       <-- CertificateWatcher

your_skill_package/
    watcher.py         <-- NEW SkillWatcher + SkillHandler
```

The lifecycle would then be:

```
App startup
     │
     ├── Initial registration
     │       └── scan user skill directory
     │
     └── SkillWatcher
             │
             ├── SKILL.md created
             │       └── register skill
             │
             ├── SKILL.md modified
             │       └── re-register skill
             │
             ├── SKILL.md moved/replaced
```

```
│       └── re-register skill
│
└── SKILL.md deleted
        └── deregister skill
```

And that last distinction is important: **deleted skills should normally be deregistered, not re-registered.**

---

1. Create a `SkillWatcher`

I'd keep it very similar to your certificate implementation, but simpler because you don't need `FilepathWatcher`.

Something like:

```python
import asyncio
from pathlib import Path

from watchdog.events import FileSystemEvent, FileSystemEventHandler

from aidl.filesystem.directory import DirectoryHandler, DirectoryWatcher


class SkillHandler(DirectoryHandler, FileSystemEventHandler):
    """
    Watch for changes to SKILL.md files.
    """

    def __init__(self, queue: asyncio.Queue, skills_directory: str):
        DirectoryHandler.__init__(self, queue=queue)
        FileSystemEventHandler.__init__(self)

        self.skills_directory = Path(skills_directory).resolve()

    def _is_skill_file(self, path: str) -> bool:
        return Path(path).name == "SKILL.md"

    def _queue_skill_event(self, action: str, path: str):
        if not self._is_skill_file(path):
            return

        self.add_to_queue({
            "action": action,
```

```
            "path": str(Path(path).resolve())
        })

    def on_created(self, event: FileSystemEvent):
        if not event.is_directory:
            self._queue_skill_event("created", event.src_path)

    def on_modified(self, event: FileSystemEvent):
        if not event.is_directory:
            self._queue_skill_event("modified", event.src_path)

    def on_deleted(self, event: FileSystemEvent):
        if not event.is_directory:
            self._queue_skill_event("deleted", event.src_path)

    def on_moved(self, event: FileSystemEvent):
        if event.is_directory:
            return

        # Old skill may have disappeared.
        self._queue_skill_event("deleted", event.src_path)

        # New location may now contain the skill.
        self._queue_skill_event("created", event.dest_path)


class SkillWatcher(DirectoryWatcher):

    def __init__(
        self,
        skills_directory: str,
        queue: asyncio.Queue
    ):
        self.skills_directory = str(
            Path(skills_directory).resolve()
        )

        super().__init__(
            handler=SkillHandler,
            queue=queue,
            skills_directory=self.skills_directory
        )

    def start(self):
        super().start(self.skills_directory)
```

Now your application can simply do:

```
watcher = SkillWatcher(
    skills_directory="/home/user/skills",
    queue=queue
)

watcher.start()
```

Since your existing `DirectoryWatcher.start()` already does:

```
Python
```

this will watch:

```
/home/user/skills/
    skill_a/SKILL.md
    skill_b/SKILL.md
    nested/skill_c/SKILL.md
    new_skill_created_later/SKILL.md
```

without restarting anything.

---

2. I would only watch `SKILL.md` initially

This keeps the implementation much cleaner.

For example:

```
skills/
    calculator/
        SKILL.md
        helper.py
        examples.txt
```

Your registry appears primarily concerned with things such as:

```
name
description
```

```
content
namespace
owners
status
```

which come from `SKILL.md`.

So initially I would say:

```
helper.py modified
        ↓
ignore

SKILL.md modified
        ↓
re-register
```

If later the registry needs to know about every resource belonging to a skill, you can change the watcher to treat **any file beneath the skill directory** as a skill update.

But I wouldn't add that complexity yet.

---

3. Separate local-skill registration from your existing GitLab registration

This is probably the biggest change to your current `skill.py`.

Right now:

```
async def register_skills(
    registry_url,
    query,
    user_env,
    token_key
)
```

is fundamentally:

> Find skills in GitLab and register them.

You now have a second source:

Find skills in the user's local filesystem and register them.

I would **not cram both behaviors into** `register_skills()`.

Instead, I'd move toward:

```
async def register_repository_skills(...):
    ...
```

and:

```
async def register_local_skill(
    registry_url: str,
    skill_path: str
):
    ...
```

plus:

```
async def deregister_local_skill(
    registry_url: str,
    skill_path: str
):
    ...
```

That makes the watcher very straightforward.

---

4. Your event processor becomes simple

Using the exact same pattern as your certificate watcher:

```
async def process_skill_event(
    event: dict,
    registry_url: str
):
```

```
    action = event["action"]
    path = event["path"]

    if action in {
        "created",
        "modified"
    }:
        await register_local_skill(
            registry_url=registry_url,
            skill_path=path
        )

    elif action == "deleted":
        await deregister_local_skill(
            registry_url=registry_url,
            skill_path=path
        )
```

Your existing `DirectoryWatcher.watch()` can already run this:

```
task = asyncio.create_task(
    watcher.watch(
        process_skill_event,
        registry_url=registry_url
    )
)

watcher.start()
```

So you are **reusing almost all of your async/event architecture**.

---

5. The local registration function can reuse much of `skill.py`

You already have this part:

Python Run

And your existing construction of:

Python Run

and:

Python Run

can also be reused.

The difference is where the data originates.

Instead of:

Python Run

you'd have something like:

```python
def extract_local_skill(skill_path: str):
    path = Path(skill_path)

    content = path.read_text(encoding="utf-8")

    metadata = parse_skill_content(content)

    return content, metadata
```

Then:

```python
async def register_local_skill(
    registry_url: str,
    skill_path: str
):

    content, metadata = extract_local_skill(skill_path)

    is_valid, error = validate_skill(
        metadata,
```

```
        content
    )

    skill_directory = Path(skill_path).parent

    skill = Skill(
        content=content,
        description=metadata.get(
            "description",
            "Nothing specified."
        ),
        group="user",
        health_url=None,
        name=metadata.get(
            "name",
            skill_directory.name
        ),
        namespace=skill_directory.name,
        owners=["User"]
    )

    registrants = await send_registration(
        registry_url=registry_url,
        registrants=[
            Registrant(
                enables=skill.description,
                error=error,
                is_active=is_valid,
                is_core=False,
                is_enabled=is_valid,
                namespace=skill.namespace,
                registration_key=skill.namespace,
                registry_location=str(skill_path),
                registry_type=RegistryType.SKILL,
                source=skill,
                status=(
                    SkillStatus.AVAILABLE
                    if is_valid
                    else SkillStatus.UNAVAILABLE
                )
            )
        ]
    )

    return [
        Registrant(**r)
        for r in registrants
    ]
```

I'm leaving the exact metadata parsing out because your current:

Python Run

isn't actually parsing `SKILL.md` frontmatter yet.

But structurally, that's where it goes.

---

6. Updating is actually easy if registration is an upsert

This is one thing I'd check in your registry implementation.

Ideally:

Python Run

uniquely identifies the skill.

Then registering:

```
calculator
```

for the first time:

```
registration_key = calculator
        ↓
INSERT
```

and registering it again after changing `SKILL.md` :

```
registration_key = calculator
        ↓
UPDATE existing registration
```

In database terminology, you want an **upsert**.

Then your watcher doesn't actually care whether the event was:

```
created
```

or:

```
modified
```

Both can simply call:

```python

```

That's a very nice property of this architecture.

---

7. Deletion is the one operation your current code doesn't show

You currently have:

```python

```

but I don't see the inverse operation in the code you've shown.

You'll need something conceptually like:

```python
await remove_registration(
    registry_url=registry_url,
    registration_key=...
)
```

or perhaps your registry has a way to set:

```python
is_active=False
status=...
```

I wouldn't invent a new registry API until we look at your existing registry code.

But conceptually:

```
SKILL.md deleted
       ↓
determine registration_key from path
       ↓
remove/deactivate that registrant
```

Notice why having a **deterministic registration key** matters.

For example, given:

```
/home/bob/skills/file_data_creator/SKILL.md
```

you could consistently use:

```
registration_key = "file_data_creator"
```

Then even after the file has been deleted, the watcher can derive:

```
‹/› Python                                                    Run
```

and knows what registry entry to remove.

You don't need the deleted file's contents.

---

8. Startup should also perform an initial scan

This is important.

The watcher should handle:

changes occurring **after** the app starts.

But at application startup you should first discover skills that already exist:

```
startup
   │
   ├── scan /user/skills
   │
```

```
    ├── register existing skills
    │
    └── start watcher
```

Something like:

```python
async def register_existing_skills(
    registry_url: str,
    skills_directory: str
):

    root = Path(skills_directory)

    for skill_file in root.rglob("SKILL.md"):
        await register_local_skill(
            registry_url=registry_url,
            skill_path=str(skill_file)
        )
```

Then start the watcher.

That gives you:

```
Existing skills       → registered at startup
New skills            → registered automatically
Modified skills       → registered again/upserted
Deleted skills        → removed/deactivated
```

Which is essentially a **live registry synchronization system**.

One change I would make to your existing generic watcher

There's something in your certificate implementation that I'd fix before leaning on this architecture more heavily.

Currently:

```python
def add_to_queue(self, event: Event):
    self.queue.put_nowait(event)
```

`watchdog.Observer` dispatches filesystem events from its own thread, while `asyncio.Queue` is **not thread-safe**.

I'd change your generic `DirectoryHandler` to bridge into the asyncio event loop safely:

```
class DirectoryHandler(FileSystemEventHandler):

    def __init__(
        self,
        queue: asyncio.Queue,
        loop: asyncio.AbstractEventLoop
    ):
        self.queue = queue
        self.loop = loop

    def add_to_queue(self, event):
        self.loop.call_soon_threadsafe(
            self.queue.put_nowait,
            event
        )
```

Then during async startup:

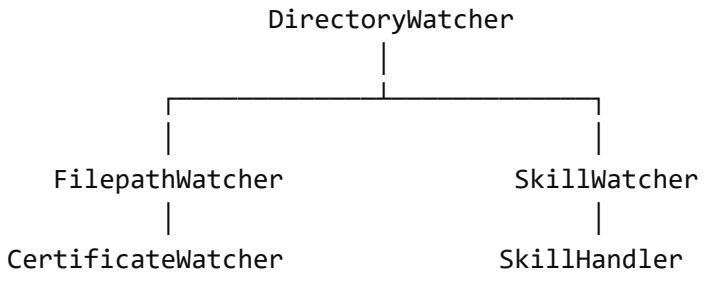
```
Python
```

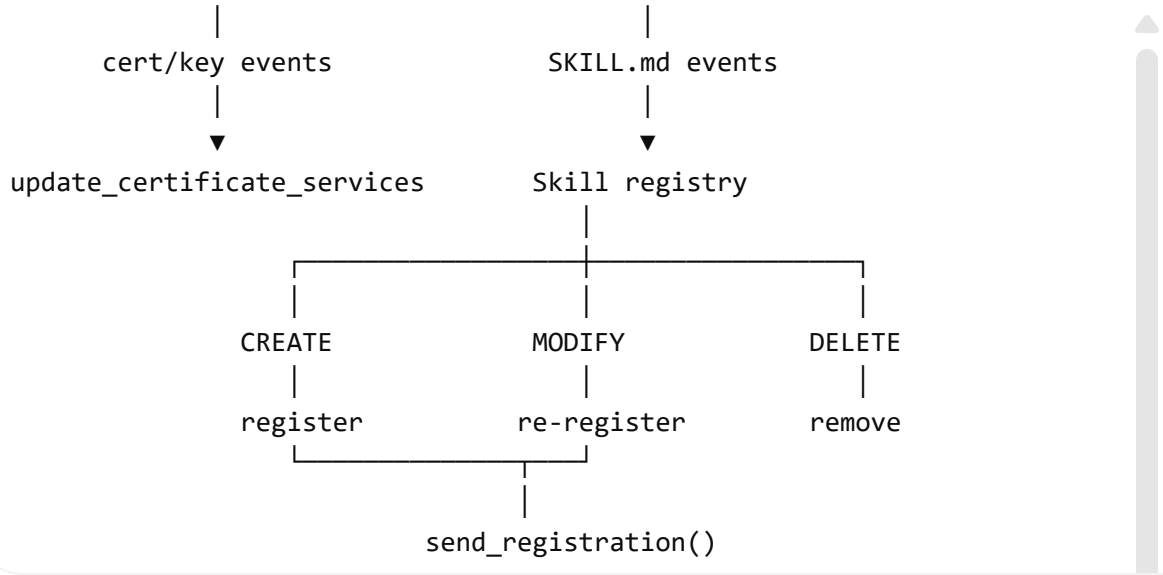
and pass that to the watcher.

That improves both your **certificate watcher and new skill watcher**.

What I'd end up with

I wouldn't redesign your existing system. You're actually already pretty close.





So **yes, I would absolutely reuse the existing watcher infrastructure**. I would add a sibling `SkillWatcher`, add local-skill registration functions to `skill.py`, and add a registry removal/deactivation operation for deleted skills.

The nice part is that this also gets you very close to the behavior you've been wanting for your Deep Agents application: a user can drop a new skill folder into their skills directory and, without restarting the application, the registry can know about it on the next filesystem event.

Sources

can you give me all the code please

Worked for 55s

Yes. I would keep your existing `directory.py` and `filepath.py` as-is for now and add the skill-specific pieces alongside them.

The structure I would use is:

```
aidl/
├── filesystem/
│   ├── directory.py              # existing
│   ├── filepath.py               # existing
│   └── skill.py                  # NEW
│
├── registry/
```

```
│   ├── base.py                  # existing
│   └── skill.py                 # UPDATED
│
caiapi/
└── lifespan/
    └── skill.py                 # NEW
```

The resulting behavior is:

```
/users/me/skills/
    calculator/
        SKILL.md

        ↓ modify SKILL.md

watchdog
   ↓
SkillHandler
   ↓
asyncio queue
   ↓
process_skill_event()
   ↓
register_local_skill()
   ↓
send_registration()
   ↓
existing registry entry updated
```

And deletion does:

```
delete calculator/SKILL.md
        ↓
watcher detects delete
        ↓
same registration_key
        ↓
send_registration()
        ↓
is_active=False
is_enabled=False
status=UNAVAILABLE
```

1. New `aidl/filesystem/skill.py`

```python
from __future__ import annotations

import asyncio
import time
from pathlib import Path

from watchdog.events import FileSystemEvent

from aidl.filesystem.directory import DirectoryHandler, DirectoryWatcher


class SkillHandler(DirectoryHandler):
    """
    Watch for changes to SKILL.md files inside a skills directory.
    """

    def __init__(
        self,
        queue: asyncio.Queue,
        skills_directory: str,
        loop: asyncio.AbstractEventLoop,
    ):
        super().__init__(queue=queue)

        self.skills_directory = Path(skills_directory).expanduser().resolve()
        self.loop = loop

        # Track existing skills so directory deletes/moves can also
        # be associated with their SKILL.md files.
        self.known_skill_files = {
            str(path.resolve())
            for path in self.skills_directory.rglob("SKILL.md")
        }

        # Filesystem watchers can emit several events for one save.
        # This prevents very rapid duplicate registrations.
        self._last_events: dict[tuple[str, str], float] = {}
        self._debounce_seconds = 0.25

    def _is_skill_file(self, path: str | Path) -> bool:
        """
        Determine whether a path points to a SKILL.md file.
        """
        return Path(path).name == "SKILL.md"

    def _add_skill_event(self, action: str, path: str | Path):
```

```
        """
        Add a skill filesystem event to the async queue.
        """

        path = str(Path(path).expanduser().resolve())

        if not self._is_skill_file(path):
            return

        event_key = (action, path)
        now = time.monotonic()

        last_event = self._last_events.get(event_key)

        if (
            last_event is not None
            and now - last_event < self._debounce_seconds
        ):
            return

        self._last_events[event_key] = now

        if action == "upsert":
            self.known_skill_files.add(path)

        elif action == "delete":
            self.known_skill_files.discard(path)

        event = {
            "action": action,
            "path": path,
        }

        # watchdog runs callbacks from another thread.
        # asyncio.Queue is not thread-safe, so hand the operation
        # back to the asyncio event loop.
        self.loop.call_soon_threadsafe(
            self.queue.put_nowait,
            event,
        )

    def on_created(self, event: FileSystemEvent):
        """
        Handle creation of a new SKILL.md.
        """

        if event.is_directory:
            return
```

```
            self._add_skill_event(
                action="upsert",
                path=event.src_path,
            )

        def on_modified(self, event: FileSystemEvent):
            """
            Handle modification of an existing SKILL.md.
            """

            if event.is_directory:
                return

            self._add_skill_event(
                action="upsert",
                path=event.src_path,
            )

        def on_deleted(self, event: FileSystemEvent):
            """
            Handle deletion of a SKILL.md or an entire skill directory.
            """

            if not event.is_directory:
                self._add_skill_event(
                    action="delete",
                    path=event.src_path,
                )
                return

            deleted_directory = Path(event.src_path).expanduser().resolve()

            # If an entire skill directory disappeared, find any known
            # SKILL.md files that used to live underneath it.
            for skill_path in list(self.known_skill_files):
                path = Path(skill_path)

                try:
                    path.relative_to(deleted_directory)
                except ValueError:
                    continue

                self._add_skill_event(
                    action="delete",
                    path=path,
                )

        def on_moved(self, event: FileSystemEvent):
            """
```

```
        Handle files/directories being renamed or atomically replaced.
        """

        if not event.is_directory:

            # Old SKILL.md path disappeared.
            if self._is_skill_file(event.src_path):
                self._add_skill_event(
                    action="delete",
                    path=event.src_path,
                )

            # New SKILL.md path appeared.
            if self._is_skill_file(event.dest_path):
                self._add_skill_event(
                    action="upsert",
                    path=event.dest_path,
                )

            return

        source_directory = Path(event.src_path).expanduser().resolve()
        destination_directory = Path(event.dest_path).expanduser().resolve()

        # Handle an entire skill directory being renamed/moved.
        matching_skills = []

        for skill_path in list(self.known_skill_files):
            path = Path(skill_path)

            try:
                relative_path = path.relative_to(source_directory)
            except ValueError:
                continue

            matching_skills.append(
                (
                    path,
                    destination_directory / relative_path,
                )
            )

        for old_path, new_path in matching_skills:

            self._add_skill_event(
                action="delete",
                path=old_path,
            )
```

```
            self._add_skill_event(
                action="upsert",
                path=new_path,
            )


class SkillWatcher(DirectoryWatcher):
    """
    Watch a user skills directory for SKILL.md changes.
    """

    def __init__(
        self,
        skills_directory: str,
        queue: asyncio.Queue,
        loop: asyncio.AbstractEventLoop,
    ):
        self.skills_directory = Path(
            skills_directory
        ).expanduser().resolve()

        super().__init__(
            handler=SkillHandler,
            queue=queue,
            skills_directory=str(self.skills_directory),
            loop=loop,
        )

    def start(self):
        """
        Start recursively watching the skills directory.
        """

        self.skills_directory.mkdir(
            parents=True,
            exist_ok=True,
        )

        self.observer.schedule(
            self.handler,
            path=str(self.skills_directory),
            recursive=True,
        )

        self.observer.start()
```

The important difference from `FilepathWatcher` is:

Python Run

You're watching the entire user skill root.

So this:

```
skills/
    existing_skill/
        SKILL.md
```

can later become:

```
skills/
    existing_skill/
        SKILL.md

    brand_new_skill/
        SKILL.md
```

and the watcher automatically sees the new directory.

---

2. Updated `aidl/registry/skill.py`

This keeps your existing GitLab registration and adds local filesystem registration.

```python
from __future__ import annotations

import os
from pathlib import Path

from dotenv import load_dotenv

from aidl import configuration
from aidl.connectors.gitlab.graphql import GitlabGraphQL
from aidl.models.api.agent import Skill, SkillAuthor
from aidl.models.api.registry import Registrant
from aidl.models.enum import RegistryType, SkillStatus
from aidl.registry.base import send_registration
```

```
def SKILL_CONTENT(
    path: str,
    group: str,
    project: str,
) -> str:
    return f"""
    query {{
        project(fullPath: "{group}/{project}") {{
            repository {{
                blobs(
                    ref: "main",
                    paths: ["{path}/SKILL.md"]
                ) {{
                    nodes {{
                        name
                        rawBlob
                    }}
                }}
            }}
        }}
    }}
    """


def determine_authorship(project: dict) -> SkillAuthor:
    """
    Extract last editor metadata.

    Args:
        project: key/value pairs representing a SKILL container.

    Returns:
        Skill author information.
    """

    return SkillAuthor(
        email="blah@bleh",
        name="Some Person",
    )


def extract_content(
    connection: GitlabGraphQL,
) -> str:
    """
    Extract SKILL.md content from repository storage.

    Args:
        connection: repository connector
```

```
    Returns:
        SKILL.md content.
    """

    return """
---
name: file_data_creator
description: Create and save file data to virtual or local filesystems.
---

### Instructions:
1. When asked to create file data, use the specific backend tool provided.
2. Ensure all data is saved with correct encoding (default to utf-8).
3. Confirm the file path before saving.
"""


def extract_skills(
    query: str,
    token: str,
) -> list[tuple]:
    """
    Extract skills from GitLab storage.

    Returns:
        Tuples containing:

            (
                content,
                author,
                metadata,
                project
            )
    """

    gg = GitlabGraphQL(
        token=token,
    )

    result = []

    for project in gg.paginate(
        query,
        ["projects"],
    ):
        content = extract_content(
            connection=gg,
        )
```

```
        author = determine_authorship(
            project,
        )

        metadata = parse_content(
            content,
            project,
        )

        result.append(
            (
                content,
                author,
                metadata,
                project,
            )
        )

    return result


def parse_content(
    content: str,
    project: dict,
) -> dict:
    """
    Extract metadata from repository SKILL content.
    """

    return {
        "description": project["description"],
        "name": project["name"],
    }


async def register_skills(
    registry_url: str,
    query: str,
    user_env: str,
    token_key: str | None = None,
) -> list[Registrant]:
    """
    Register repository skills with the gateway.
    """

    # Reload .env in case it changed after application startup.
    load_dotenv(
        user_env,
```

```
        override=True,
    )

    if not token_key:
        return []

    token = os.getenv(
        token_key,
    )

    if token is None:
        return []

    skills = [
        (
            validate_skill(
                metadata,
                content,
            ),
            user_enabled(),
            Skill(
                content=content,
                description=metadata.get(
                    "description",
                    "Nothing specified.",
                ),
                group=project["fullPath"],
                health_url="fill in from project",
                name=metadata.get(
                    "name",
                    "Unknown",
                ),
                namespace=project["fullPath"],
                owners=[
                    (
                        f"{configuration.BRAND_NAME} Team"
                        if project["fullPath"]
                        in configuration.SKILL_REPOSITORY_CORE_PATH
                        else (
                            author.name
                            if author
                            else "Unknown"
                        )
                    )
                ],
            ),
        )
        for content, author, metadata, project
        in extract_skills(query, token)
```

```
    ]

    registrants = await send_registration(
        registry_url=registry_url,
        registrants=[
            Registrant(
                enables=skill.description,
                error=error,
                is_active=is_valid,
                is_core=(
                    skill.group
                    in configuration.SKILL_REPOSITORY_CORE_PATH
                ),
                is_enabled=(
                    is_valid
                    and is_enabled
                ),
                namespace=skill.group,
                registration_key=skill.namespace,
                registry_location=(
                    f"{skill.group}/"
                    f"{skill.name}/"
                    "SKILL.md"
                ),
                registry_type=RegistryType.SKILL,
                source=skill,
                status=(
                    SkillStatus.AVAILABLE
                    if is_valid
                    else SkillStatus.UNAVAILABLE
                ),
            )
            for (
                (is_valid, error),
                is_enabled,
                skill,
            )
            in skills
        ],
    )

    return [
        Registrant(**registrant)
        for registrant in registrants
    ]


# ---------------------------------------------------------------------
# Local user skills
```

```
# ----------------------------------------------------------------------


def parse_local_skill_content(
    content: str,
) -> dict:
    """
    Parse the simple YAML frontmatter from a local SKILL.md.

    Expected structure:

        ---
        name: calculator
        description: Perform calculations.
        ---

        Instructions...
    """

    lines = content.splitlines()

    if not lines:
        return {}

    if lines[0].strip() != "---":
        return {}

    metadata = {}

    for line in lines[1:]:

        line = line.strip()

        if line == "---":
            break

        if not line:
            continue

        if line.startswith("#"):
            continue

        if ":" not in line:
            continue

        key, value = line.split(
            ":",
            1,
        )
```

```
        key = key.strip()
        value = value.strip()

        # Remove simple surrounding quotes.
        if (
            len(value) >= 2
            and value[0] == value[-1]
            and value[0] in {"'", '"'}
        ):
            value = value[1:-1]

        metadata[key] = value

    return metadata


def get_local_skill_identity(
    skill_path: str,
    skills_directory: str,
) -> tuple[str, str, str]:
    """
    Determine the stable registry identity for a local skill.

    Returns:
        (
            registration_key,
            namespace,
            default_name
        )
    """

    skill_file = Path(
        skill_path
    ).expanduser().resolve()

    skills_root = Path(
        skills_directory
    ).expanduser().resolve()

    skill_directory = skill_file.parent

    try:
        relative_directory = skill_directory.relative_to(
            skills_root
        )

        namespace = relative_directory.as_posix()
```

```
    except ValueError:
        # Fallback if somebody passes a SKILL.md outside
        # of the configured root.
        namespace = skill_directory.name

    if namespace in {"", "."}:
        namespace = skill_directory.name

    default_name = skill_directory.name

    # Prefix local skills so they do not collide with
    # repository-backed registration keys.
    registration_key = f"local:{namespace}"

    return (
        registration_key,
        namespace,
        default_name,
    )


def extract_local_skill(
    skill_path: str,
) -> tuple[str, dict]:
    """
    Read a local SKILL.md and extract its metadata.
    """

    skill_file = Path(
        skill_path
    ).expanduser().resolve()

    content = skill_file.read_text(
        encoding="utf-8",
    )

    metadata = parse_local_skill_content(
        content,
    )

    return (
        content,
        metadata,
    )


def build_local_skill(
    skill_path: str,
    skills_directory: str,
```

```
) -> tuple[
    tuple[bool, str | None],
    Registrant,
]:
    """
    Build a registry object for a local skill.
    """

    content, metadata = extract_local_skill(
        skill_path,
    )

    registration_key, namespace, default_name = (
        get_local_skill_identity(
            skill_path,
            skills_directory,
        )
    )

    is_valid, error = validate_skill(
        metadata,
        content,
    )

    skill = Skill(
        content=content,
        description=metadata.get(
            "description",
            "Nothing specified.",
        ),
        group="local",
        health_url="",
        name=metadata.get(
            "name",
            default_name,
        ),
        namespace=namespace,
        owners=["User"],
    )

    registrant = Registrant(
        enables=skill.description,
        error=error,
        is_active=is_valid,
        is_core=False,
        is_enabled=(
            is_valid
            and user_enabled()
        ),
```

```
            namespace=namespace,
            registration_key=registration_key,
            registry_location=str(
                Path(skill_path).expanduser().resolve()
            ),
            registry_type=RegistryType.SKILL,
            source=skill,
            status=(
                SkillStatus.AVAILABLE
                if is_valid
                else SkillStatus.UNAVAILABLE
            ),
        )

        return (
            (is_valid, error),
            registrant,
        )


async def register_local_skill(
    registry_url: str,
    skill_path: str,
    skills_directory: str,
) -> list[Registrant]:
    """
    Register or re-register a local SKILL.md.

    Sending the same registration_key again is expected
    to update the existing registry entry.
    """

    skill_file = Path(
        skill_path
    ).expanduser().resolve()

    # A delete could occur between receiving the filesystem
    # event and processing it.
    if not skill_file.exists():
        return await register_deleted_local_skill(
            registry_url=registry_url,
            skill_path=skill_path,
            skills_directory=skills_directory,
        )

    _, registrant = build_local_skill(
        skill_path=skill_path,
        skills_directory=skills_directory,
    )
```

```
    result = await send_registration(
        registry_url=registry_url,
        registrants=[
            registrant,
        ],
    )

    return [
        Registrant(**item)
        for item in result
    ]


async def register_deleted_local_skill(
    registry_url: str,
    skill_path: str,
    skills_directory: str,
) -> list[Registrant]:
    """
    Re-register a deleted local skill as unavailable.

    The important part is that this uses the exact same
    registration_key that the skill used before deletion.
    """

    registration_key, namespace, default_name = (
        get_local_skill_identity(
            skill_path,
            skills_directory,
        )
    )

    skill = Skill(
        content="Local skill has been deleted.",
        description="Local skill has been deleted.",
        group="local",
        health_url="",
        name=default_name,
        namespace=namespace,
        owners=["User"],
    )

    registrant = Registrant(
        enables=skill.description,
        error="SKILL.md was deleted from the local filesystem.",
        is_active=False,
        is_core=False,
        is_enabled=False,
```

```
            namespace=namespace,
            registration_key=registration_key,
            registry_location=str(
                Path(skill_path).expanduser().resolve()
            ),
            registry_type=RegistryType.SKILL,
            source=skill,
            status=SkillStatus.UNAVAILABLE,
        )

        result = await send_registration(
            registry_url=registry_url,
            registrants=[
                registrant,
            ],
        )

        return [
            Registrant(**item)
            for item in result
        ]


async def register_existing_local_skills(
    registry_url: str,
    skills_directory: str,
) -> list[Registrant]:
    """
    Register every local skill already present at startup.
    """

    skills_root = Path(
        skills_directory
    ).expanduser().resolve()

    skills_root.mkdir(
        parents=True,
        exist_ok=True,
    )

    result: list[Registrant] = []

    for skill_file in skills_root.rglob(
        "SKILL.md"
    ):
        registrants = await register_local_skill(
            registry_url=registry_url,
            skill_path=str(skill_file),
            skills_directory=str(skills_root),
```

```
        )

        result.extend(
            registrants
        )

    return result


async def process_local_skill_event(
    event: dict,
    registry_url: str,
    skills_directory: str,
):
    """
    Process an event generated by SkillWatcher.
    """

    action = event["action"]
    path = event["path"]

    if action == "upsert":

        await register_local_skill(
            registry_url=registry_url,
            skill_path=path,
            skills_directory=skills_directory,
        )

        return

    if action == "delete":

        await register_deleted_local_skill(
            registry_url=registry_url,
            skill_path=path,
            skills_directory=skills_directory,
        )


def validate_skill(
    metadata: dict,
    content: str,
) -> tuple[bool, str | None]:
    """
    Basic SKILL.md validation.
    """

    if not content.strip():
```

```
        return (
            False,
            "SKILL.md is empty.",
        )

    if not metadata.get("name"):
        return (
            False,
            "SKILL.md does not contain a name.",
        )

    if not metadata.get("description"):
        return (
            False,
            "SKILL.md does not contain a description.",
        )

    return (
        True,
        None,
    )


def user_enabled() -> bool:
    """
    Determine whether user skills are enabled.
    """

    return True
```

There are a couple of deliberate changes to your existing code here.

I fixed:

Python Run

because that was passing the **function itself** instead of the extracted string.

It's now:

```
content = extract_content(
    connection=gg,
)
```

```
metadata = parse_content(
    content,
    project,
)
```

I also changed:

```
‹/› Python                                                    Run
```

to:

```
load_dotenv(
    user_env,
    override=True,
)
```

because your comment says:

> reload `.env` in case edited after app init

Without `override=True`, an existing environment variable won't normally be replaced.

---

3. New `caiapi/lifespan/skill.py`

This is analogous to your existing certificate startup code.

```
from __future__ import annotations

import asyncio

from aidl.filesystem.skill import SkillWatcher
from aidl.registry.skill import (
    process_local_skill_event,
    register_existing_local_skills,
)

from caiapi.lifespan.task import handle_task_error
```

```
async def startup(
    registry_url: str,
    skills_directory: str,
) -> tuple[SkillWatcher, asyncio.Task]:
    """
    Start local skill discovery and filesystem monitoring.

    Existing skills are registered at startup.

    Future SKILL.md creation, modification, movement,
    and deletion are handled automatically.
    """

    queue = asyncio.Queue()

    loop = asyncio.get_running_loop()

    watcher = SkillWatcher(
        skills_directory=skills_directory,
        queue=queue,
        loop=loop,
    )

    # Start the async queue consumer.
    task = asyncio.create_task(
        watcher.watch(
            process_local_skill_event,
            registry_url=registry_url,
            skills_directory=skills_directory,
        ),
        name="skill-watch-handler",
    )

    task.add_done_callback(
        handle_task_error
    )

    # Start filesystem watching BEFORE the initial scan.
    #
    # This prevents a small race where somebody could add a
    # skill between the initial scan and watcher startup.
    watcher.start()

    # Register anything already present.
    await register_existing_local_skills(
        registry_url=registry_url,
        skills_directory=skills_directory,
    )
```

```
    return (
        watcher,
        task,
    )


async def shutdown(
    watcher: SkillWatcher,
    task: asyncio.Task,
):
    """
    Shut down local skill filesystem monitoring.
    """

    watcher.stop()

    task.cancel()

    try:
        await task

    except asyncio.CancelledError:
        pass
```

---

4. How to connect this to your application's lifespan

Suppose you already have something like:

```
@asynccontextmanager
async def lifespan(app: FastAPI):

    # startup

    yield

    # shutdown
```

You would add:

```
from caiapi.lifespan.skill import (
    startup as skill_startup,
```

```
    shutdown as skill_shutdown,
```

and then:

Python Run

```
@asynccontextmanager
async def lifespan(app: FastAPI):

    skill_watcher, skill_task = await skill_startup(
        registry_url=app.state.registry_url,
        skills_directory=app.state.skills_directory,
    )

    try:
        yield

    finally:
        await skill_shutdown(
            watcher=skill_watcher,
            task=skill_task,
        )
```

If you get the directory from configuration instead:

Python Run

```
skill_watcher, skill_task = await skill_startup(
    registry_url=configuration.REGISTRY_URL,
    skills_directory=configuration.USER_SKILLS_DIRECTORY,
)
```

What happens with this implementation

Suppose you start with:

```
/home/selvyn/skills/
```

and it's empty.

The watcher is still running because it watches:

```
/home/selvyn/skills/
```

not individual skill paths.

Then you create:

```
/home/selvyn/skills/
    calculator/
        SKILL.md
```

with:

Markdown

```
---
name: calculator
description: Perform mathematical calculations.
---

Use the calculator tools when mathematical operations are needed.
```

`watchdog` produces:

```
created:
    /home/selvyn/skills/calculator/SKILL.md
```

The handler converts that into:

Python Run

```
{
    "action": "upsert",
    "path": "/home/selvyn/skills/calculator/SKILL.md",
}
```

The async worker receives it and calls:

Python Run

```
await register_local_skill(...)
```

which builds:

```
name:
```

```
calculator

namespace:
calculator

registration_key:
local:calculator

status:
AVAILABLE
```

and sends it through your existing:

Python Run

```python
send_registration()
```

If you then edit:

```
calculator/SKILL.md
```

you get:

```
modified
   ↓
upsert
   ↓
registration_key = local:calculator
   ↓
send_registration()
```

Assuming your registry treats `registration_key` as the identity, it updates the existing registration.

If you delete:

```
calculator/SKILL.md
```

the watcher sends:

Python Run

```
{
    "action": "delete",
    "path": "/home/selvyn/skills/calculator/SKILL.md",
}
```

and we deliberately calculate the exact same:

```
registration_key:
local:calculator
```

but send:

Python Run

```
is_active=False
is_enabled=False
status=SkillStatus.UNAVAILABLE
error="SKILL.md was deleted from the local filesystem."
```

So your registry becomes:

```
calculator

status: UNAVAILABLE
active: false
enabled: false
```

instead of leaving a stale skill that Deep Agents might think still exists.

One thing I would verify next

The only piece I can't verify from the code you've given me is exactly how:

Python Run

```
send_registration()
```

handles this:

```
registration_key = local:calculator
```

being sent twice.

The architecture above assumes:

```
first registration
local:calculator
      ↓
INSERT


second registration
local:calculator
      ↓
UPDATE
```

rather than:

```
second registration
      ↓
duplicate registration error
```

If you show me your `send_registration()` implementation and the registry endpoint it calls, I can tell you whether the **upsert/re-registration portion is already supported or whether that is the one remaining change needed**.